# TENIR-Gov Operationalization Guide

**Version:** 1.0 · **Date:** 2026-10-19 **Scope:** How to deploy, configure, and operate TENIR-Gov as a runtime governance membrane for AI agents — from single-process observation to multi-host enforcement.

---

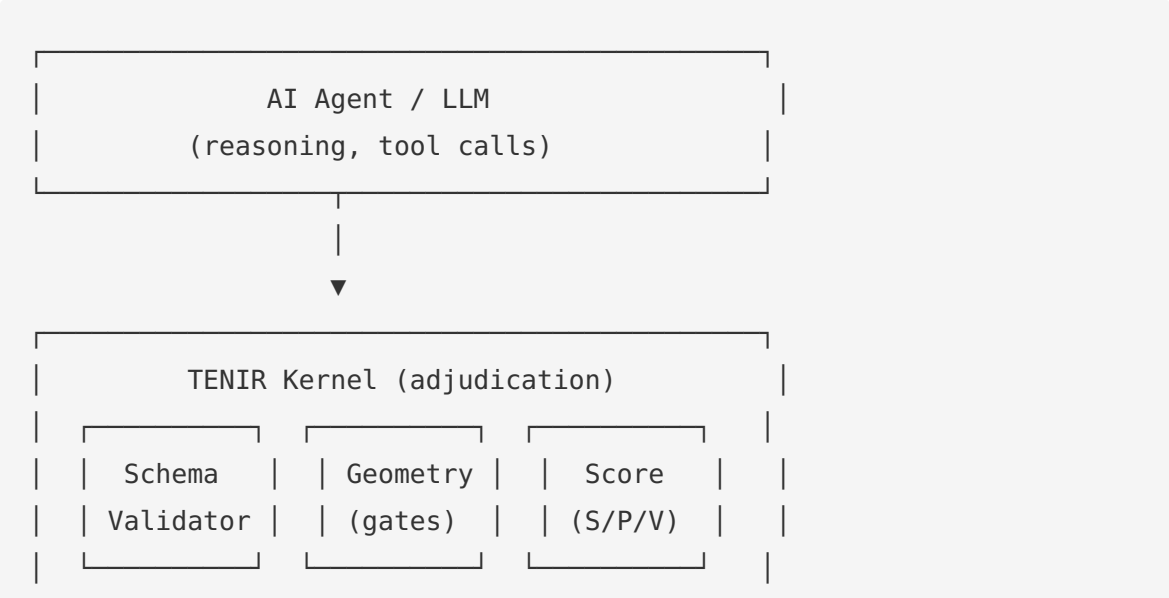
## Table of Contents

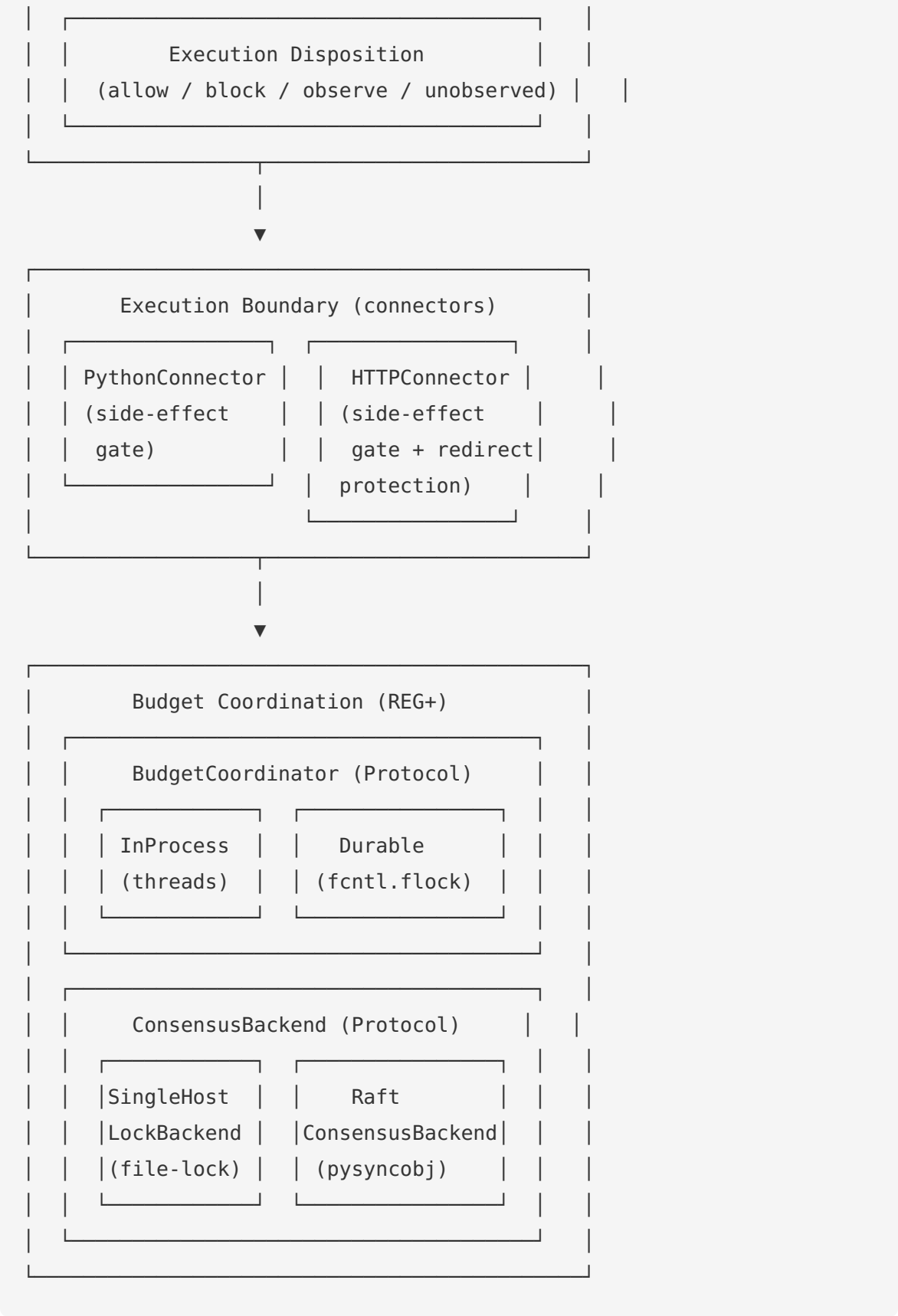

1. Architecture Overview
2. Deployment Modes
3. Execution Boundary Flow
4. Single-Host Setup (SingleHostLockBackend)
5. Multi-Host Setup (RaftConsensusBackend)
6. Known Limitations (Honest)


---

## 1. Architecture Overview

TENIR-Gov sits between an AI agent's reasoning and the real-world effects of its actions. It adjudicates, decides, and gates — it does not generate the actions itself.




Execution Disposition
(allow / block / observe / unobserved)
Execution Boundary (connectors)
PythonConnector
(side-effect
gate)
HTTPConnector
(side-effect
gate + redirect
protection)
Budget Coordination (REG+)
BudgetCoordinator (Protocol)
InProcess
(threads)
Durable
(fcntl.flock)
ConsensusBackend (Protocol)
SingleHost
LockBackend
(file-lock)
Raft
ConsensusBackend
(pysyncobj)

**The key architectural decision:** the governance membrane sits **in front of** the side effect, not after it. If the governance decision is negative, the function or HTTP request is **never invoked**.

---

# 2. Deployment Modes

TENIR-Gov supports four operating modes, from observation to enforcement:

| Mode | Evaluate | Record | Intended Block | Actual Block | Use Case |
|---|---|---|---|---|---|
| `SHADOW_OFF` | No | No | No | No | Governance bypass (explicit) |
| `SHADOW_PASSIVE` | Yes | Yes | Yes | No | Observation, data collection |
| `SHADOW_CRITICAL` | Yes | Yes | Yes | No | High-risk observation with alerts |
| `ENFORCE` | Yes | Yes | Yes | **Yes** | Production enforcement |

## The Critical Distinction

`SHADOW_OFF` returns `NOT_EVALUATED` with `execution_disposition = allow_unobserved`. This is **not** the same as `PASS` — it means governance was not invoked, not that the action was approved.

```
NOT_EVALUATED ≠ PASS
NOT_EVALUATED ≠ ADMISSIBLE
```

The tests explicitly verify this invariant: even under extreme TAU (trust threshold), `SHADOW_OFF` does not evaluate. This prevents a future implementation from implicitly re-introducing evaluation when the mode is `SHADOW_OFF`.

## Recommended Progression

1. **Start with `SHADOW_PASSIVE`** — observe what your agents do, collect data on governance decisions
2. **Move to `SHADOW_CRITICAL`** — add alerts and intended blocks for high-risk actions, but don't actually block
3. **Switch to `ENFORCE`** — only after you've validated your policies and thresholds against real observation data

---

# 3. Execution Boundary Flow

The execution boundary is where the governance decision becomes an operational gate. This is what distinguishes TENIR-Gov from a decision engine with a wrapper — the control is on the path of the effect.

## Python Connector

```
from tenir_kernel.kernel import TenirKernel
from tenir_kernel.connectors.python import PythonConnector

# 1. Create kernel with policy
kernel = TenirKernel(
    schema_version="1.2",
    operating_mode="ENFORCE",
    hard_veto_threshold=0.75,
    flag_threshold=0.90,
)

# 2. Wrap any function with the execution boundary
@PythonConnector(kernel=kernel)
def execute_trade(symbol, quantity, price):
    # This function ONLY runs if:
    # - kernel adjudication returns execution_allowed=True
    # - budget reservation succeeded (if budget is configured)
    return broker_api.place_order(symbol, quantity, price)
```

```
# 3. Call the governed function
result = execute_trade("AAPL", 100, 150.0)
# If HARD_VETO: GovernanceBlockedError raised, function never called
# If PASS: function executes, result + decision returned
```

**The key property:** the side effect (broker API call) is **never invoked** if the governance decision is negative. The connector sits between the decision and the effect, not after it.

## HTTP Connector

Same pattern, with additional redirect protection to prevent credential leakage via HTTP redirects:

```
from tenir_kernel.connectors.http import HTTPConnector

connector = HTTPConnector(kernel=kernel)

# If HARD_VETO: no HTTP request is sent
# If PASS: HTTP request sent with redirect protection active
# _RejectRedirects prevents credentials from following redirects to untrusted destina
response = connector.execute(
    method="POST",
    url="https://api.broker.com/orders",
    json={"symbol": "AAPL", "quantity": 100, "price": 150.0},
    headers={"Authorization": "Bearer ..."},
)
```

## What the Execution Boundary Guarantees

| Property | Guarantee |
|---|---|
| **No bypass** | The connector is the only path to the side effect |
| **No effect on denial** | If `execution_allowed=False`, the function/HTTP call is never made |

| Property | Guarantee |
| --- | --- |
| **Redirect protection** | HTTP credentials do not follow redirects to untrusted destinations |
| **Decision recorded** | Every adjudication produces a ledger entry (except `SHADOW_OFF`) |

### What It Does Not Guarantee

| Limitation | Explanation |
| --- | --- |
| **No permit binding** | The decision is not cryptographically bound to the payload that executes |
| **No receipt** | The ledger records the decision but does not produce a tamper-evident receipt |
| **No post-conditions** | The system does not verify that the effect actually occurred after execution |

---

## 4. Single-Host Setup (SingleHostLockBackend)

For single-host, multi-process deployments. This is the simplest conforming backend.

### Installation

```
pip install tenir-governance
```

### Configuration

```
from tenir_governance.operations.consensus_backend import SingleHostLockBackend
```

```
# Initialize with durable storage
backend = SingleHostLockBackend(
    store_dir="/var/lib/tenir/consensus",
    provider_reconcile=my_reconcile_function,  # optional
)

# Register a budget resource
backend.register_resource("api-budget", 10000)
```

## Usage

```
# Reserve budget for an operation
result = backend.reserve("op-unique-key-123", 500, "api-budget")
if result.status == "RESERVED":
    try:
        # Execute the operation
        do_something_expensive()
        # Commit the budget (transfer from reserved to consumed)
        backend.commit(result.reservation_id)
    except Exception:
        # Release the budget back to available
        backend.release(result.reservation_id)
elif result.status == "INSUFFICIENT_BUDGET":
    # Not enough budget — agent must wait or use smaller batch
    pass
elif result.status == "DUPLICATE_KEY":
    # This idempotency key already exists with a different amount
    pass
```

## Reconciliation

When an operation's outcome is unknown (e.g., network timeout), the reservation enters `UNKNOWN` state. Reconciliation queries the provider to determine the actual outcome:

```
def my_reconcile_function(key):
    """Query the external provider to determine if the effect occurred."""
```

```
    response = external_api.get_operation_status(key)
    if response.status == "COMPLETED":
        return True   # effect observed → commit
    elif response.status == "FAILED":
        return False  # no effect → release
    else:
        return None   # indeterminate → stays UNKNOWN, human review

backend = SingleHostLockBackend(
    store_dir="/var/lib/tenir/consensus",
    provider_reconcile=my_reconcile_function,
)

# Later, reconcile the uncertain operation
result = backend.reconcile("op-unique-key-123")
# → CONFIRMED (effect observed, committed)
# → FAILED (no effect, released)
# → UNRESOLVED (provider unreachable, stays UNKNOWN)
```

## What This Gives You

- Cross-process budget coordination via `fcntl.flock`
- Durable state (survives process restart via temp-file-then-rename with fsync)
- I8 enforced atomically under file lock
- C1 (safety), C2 (liveness), I8 for single-host
- Idempotency via key-based reservation
- Reconciliation for uncertain outcomes

## What It Does NOT Give You

- Cross-host coordination ( `fcntl.flock` is local to one OS)
- C3 (cross-host fault tolerance)
- Byzantine tolerance
- If the host crashes, the file lock is released but the durable state on disk is preserved

---

# 5. Multi-Host Setup (RaftConsensusBackend)

For multi-host deployments with crash-stop fault tolerance. State is replicated across all nodes via Raft consensus.

## Installation

```
pip install tenir-governance[pysyncobj]
# or
pip install tenir-governance pysyncobj
```

## Configuration

```
from tenir_governance.operations.raft_backend import RaftConsensusBackend

# Node 1 (leader or follower — Raft elects automatically)
node1 = RaftConsensusBackend(
    self_addr="10.0.1.1:5000",
    other_addrs=["10.0.1.2:5000", "10.0.1.3:5000"],
    store_dir="/var/lib/tenir/raft-node1",
    provider_reconcile=my_reconcile_function,
)

# Wait for cluster ready (leader election)
assert node1.wait_ready(timeout=10.0)

# Register resource (replicated to all nodes)
node1.register_resource("global-budget", 100000)
```

## Usage

```
# Any node can reserve (forwarded to leader if needed)
result = node1.reserve("op-key-456", 5000, "global-budget")
```

```
if result.status == "RESERVED":
    try:
        execute_operation()
        node1.commit(result.reservation_id)
    except Exception:
        node1.release(result.reservation_id)

# Check state from any node
state = node1.get_state("global-budget")
print(f"available={state.available}, reserved={state.reserved}, "
      f"committed={state.committed}, capacity={state.capacity}")
```

## Fault Tolerance

```
# If node 1 crashes, nodes 2 and 3 maintain quorum (2 of 3)
# Operations continue on the remaining nodes

# When node 1 recovers, it catches up via the Raft log:
node1 = RaftConsensusBackend(
    self_addr="10.0.1.1:5000",
    other_addrs=["10.0.1.2:5000", "10.0.1.3:5000"],
    store_dir="/var/lib/tenir/raft-node1",  # existing log is replayed
)
node1.wait_ready(timeout=10.0)
# State is now consistent with the other nodes
```

## Determinism Requirement

The Raft backend generates `reservation_id` and `timestamp` **client-side** and passes them as command arguments. This is critical: generating UUIDs or timestamps inside a `@replicated` method would produce different values on each node, violating Raft's determinism requirement and causing state divergence.

```
# This is what happens inside reserve():
reservation_id = str(uuid.uuid4())  # generated client-side
timestamp = time.time()             # generated client-side
result = self._raft.reserve(
```

```
    key, amount, resource_id, reservation_id, timestamp,
    sync=True, timeout=10.0
)
```

### What This Gives You

- Cross-host budget coordination via Raft consensus
- C3 (crash-stop fault tolerance) with `n ≥ 2f + 1`
- I8 enforced on leader before replication
- Leader failover with state preservation
- Durable Raft log (survives node restart, replays on recovery)
- Same interface as `SingleHostLockBackend` — drop-in replacement

### What It Does NOT Give You

- Byzantine fault tolerance (Raft is crash-only)
- Cryptographic authentication of cluster members (pysyncobj uses `pickle` without encryption by default)
- Tested on real separate hosts (current tests use localhost only)
- Network partition tolerance beyond Raft's built-in safety guarantees
- `get_state()` on followers is eventually consistent, not linearizable

---

## 6. Known Limitations (Honest)

These limitations are documented in the code and should be communicated to any user or auditor.

### P0 — Must Fix Before Production

1. **`reconcile_commit()` I8 violation.** When reconciling a RELEASED reservation, `reconcile_commit()` adds to `committed` without subtracting from `reserved` (because the reservation was already released). This can produce `reserved + committed > capacity`. **Confirmed bug with reproduced counter-example.**

2. **`time.time()` in `@replicated` methods.** The `commit()`, `release()`, `reconcile_commit()`, and `reconcile_release()` methods call `time.time()` to produce timestamps. In a replicated state machine, this produces different values on each node, violating determinism. Same class of bug as the `uuid.uuid4()` issue already fixed for `reserve()`.
3. **No input validation on `amount`.** Negative values, NaN, or non-numeric types can bypass the budget check. For a financial system, this is unacceptable.

## P1 — Should Fix Before Certification

1. **Non-atomic durable stores.** `budget.json` and `reservations.json` are written separately. A crash between writes can corrupt state. Should use a single transactional store (e.g., SQLite with WAL).
2. **`register_resource()` overwrites existing state.** On reload after crash, calling `register_resource()` erases the existing consumption history. Should preserve existing state or require explicit reset.
3. **Linearizability checker shortcuts.** The checker compares only `status` fields (not `reservation_id`), and the Raft tests use a simplified spec that maps any commit UUID to the first RESERVED reservation. This demonstrates the approach but does not constitute a rigorous proof.
4. **pysyncobj pickle security.** No encryption or authentication by default. Untrusted cluster members can execute arbitrary code via pickle deserialization. `SyncObjConf(password=...)` is available but not activated.
5. **Conformance harness tests 5.5.0, not 5.6.0.** The external harness (tenir-conformance-s1-g1) validates the `SingleHostLockBackend`, not the `RaftConsensusBackend`. The Raft backend's 29 tests are internal only.

## P2 — Known Scope Limits

1. **Raft tests use localhost only.** No real multi-host or network partition testing. C3 is claimed for crash-stop but not validated under real network conditions.
2. **Evidence is self-attested.** The hash proves the file is unchanged since attestation, not that the test was executed correctly. Independent attestation requires a third-party executor and cryptographic signature.

3. `deployment.manifest` **is missing.** This is a blocking dependency for external audit. Without it, the auditor cannot link evidence to the deployment hypothesis.
4. **No byzantine fault tolerance.** No backend handles byzantine faults. This is correctly declared as out of scope.
5. `TENIR_API_TOKEN` **is a shared service principal.** Not per-user IAM. Acceptable for a controlled pilot; insufficient for multi-tenant production.
6. **No permit → payload binding.** TENIR-Gov has decision → execution boundary → budget, but not the full REG Core chain (permit → payload binding → consumption → commit → receipt). This is the gap between "REG-compatible execution boundary" and "REG implementation."
7. **Documentation secondary files unreconciled.** CITATION.cff, .zenodo.json, pitch deck, and HARDENING_REPORT still contain old test counts (557/96%). These should be contextualized as historical, not deleted.